

DocFlow

Document Approval System

Project Documentation

Repository: <https://github.com/ArshdeepSinghVirdi/DocFlow>

A role-based, multi-stage document review & approval platform

Live demo: <https://doc-flow-system.vercel.app>

1. Problem Statement

In most organizations, documents such as proposals, budget requests, and policy files must pass through several rounds of review before they can be approved. When this process is handled over email, shared drives, or paper, it becomes slow and difficult to track. Common problems include:

- No single source of truth — files and feedback are scattered across inboxes and chat threads, so no one can see a document's current status at a glance.
- Unclear ownership — it is hard to know which reviewer a document is waiting on, or which stage of approval it has reached.
- Weak audit trail — there is no reliable, time-stamped history of who submitted, commented on, approved, rejected, or returned a document for revision.
- Inconsistent access control — submitters, reviewers, and administrators all need different permissions, which is difficult to enforce manually.
- Slow revision loops — when changes are requested, reconnecting the right version with the right reviewer is error-prone.

The organization needs a centralized system that lets users submit documents, routes each document through the correct sequence of approvers, enforces role-based permissions, and keeps a complete, transparent history of every action.

2. Solution Overview

The Document Approval System is a full-stack web application that digitizes the entire document review lifecycle. Users sign in and, based on their assigned role, are taken to a dedicated dashboard:

- **Submitters** upload documents (PDF, DOCX, or image), track their status, respond to feedback, and resubmit revised versions.
- **Approvers** review the documents assigned to them and take action — approve and advance to the next stage, reject, or request a revision — while leaving comments.
- **Administrators** manage users and their roles, configure approval workflows (which approver handles each stage), and monitor overall activity and approver workload.

Each document moves through a configurable, multi-stage workflow — typically an Initial level, a Compliance level, and a Final level — with a different approver responsible at each stage. Every action is recorded in a time-stamped approval history, and a comment thread keeps all feedback attached to the document itself. A backend REST API handles authentication, role-based authorization, file storage, and the approval state machine, while a React single-page application provides the user interface.

3. Technology Stack

The project is organized as a two-part repository with a separate frontend and backend.

Layer	Technology	Role in the project
Frontend	React + Vite (JavaScript)	Single-page application; role-specific dashboards, document views, forms, and status tracking. Vite provides the dev server and build tooling.
Styling	CSS	Component and layout styling for the dashboards and document pages.
Backend	Node.js + Express (REST API)	Exposes the /api/* endpoints, applies authentication and role checks, and implements the approval workflow logic. In production it also serves the compiled frontend from a single integrated server.
Authentication	Token-based (JWT Bearer)	Login issues a token plus the user's role; protected endpoints require the token in the Authorization header.
File uploads	Multipart handler (e.g. Multer)	Accepts document uploads (PDF, DOCX, image) and stores them for later download.
Database	Server-side database	Stores users, documents, workflow configurations, comments, and approval history.
Dev orchestration	concurrently	Runs the backend and frontend together during development from the repository root.
Hosting	Vercel	Public deployment of the application (live demo: doc-flow-fawn.vercel.app).
Version control	Git / GitHub	Source control and collaboration (repositories: Document_Approval and the integrated DocFlow build).

Note: The frontend (React + Vite) and the backend framework (Node.js + Express) are confirmed from the project repositories — the companion DocFlow repository describes the project as an “Integrated Document Approval System with React Vite Frontend and Express Backend.” The specific database engine and helper libraries (for example the authentication and file-upload packages) should be confirmed against the backend source and recorded here.

4. System Architecture

The application follows a classic three-tier architecture. The browser-based React client communicates with the Express REST API over HTTPS using JSON, sending a JWT token with each authenticated request. The API verifies the token and the user's role before invoking the relevant controller, which reads from or writes to the database and the file store.

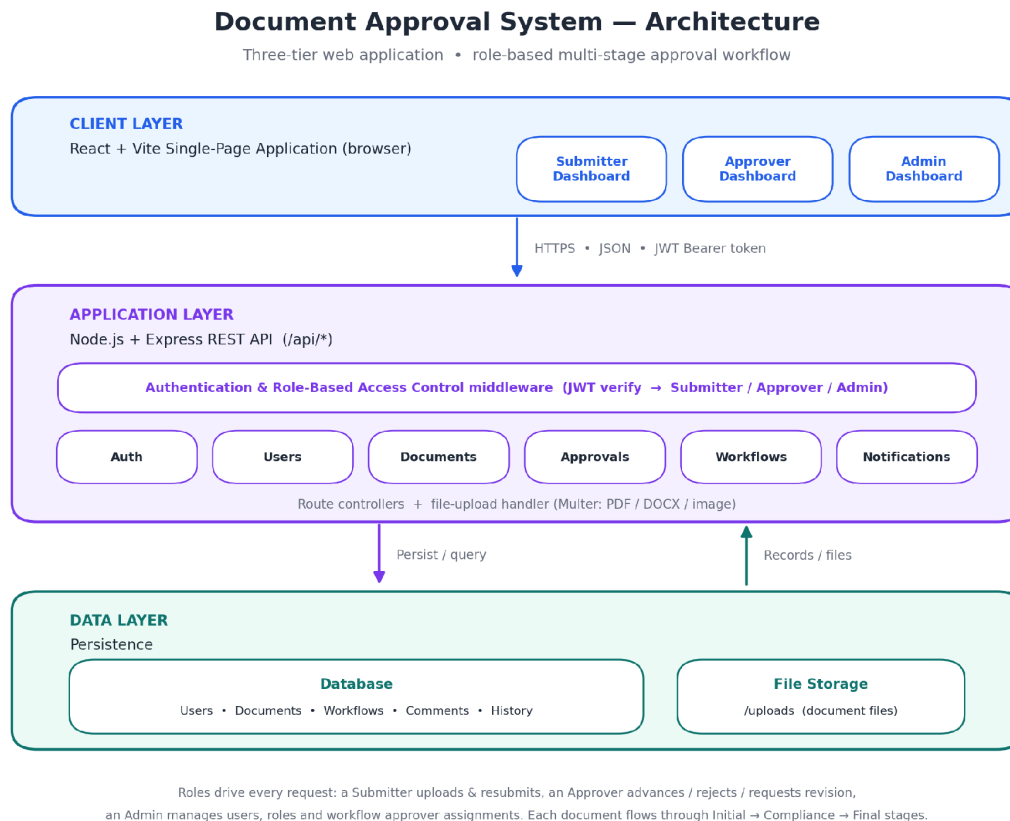


Figure 1 — System architecture (client, application, and data tiers).

Request flow

1. The user signs in; the API validates credentials and returns a JWT and the user's role.
2. The React app stores the token and sends it as a Bearer token on every subsequent request.
3. Express middleware verifies the token and checks the role before allowing access to a route.
4. Controllers run the requested operation — query documents, store an upload, advance a workflow stage — and return JSON.
5. Uploaded files are written to storage; document records, comments, and approval history are persisted in the database.

5. Feature List

Authentication & access control

- User registration and token-based login that returns the user's role.
- Three roles — Submitter, Approver, and Admin — each with a tailored dashboard and permissions.
- Session/token invalidation on logout and a “current user” endpoint for the active session.

Document submission (Submitter)

- Upload documents with a title, description, and category; supported file types include PDF, DOCX, and images.
- Personal dashboard with status counts (total, approved, pending, revision, rejected).
- Search and filter documents by status and category; download the original file.
- Resubmit a document after a revision request, optionally attaching an updated file.

Review & approval (Approver)

- See the list of documents assigned to you, with filters.
- Approve and advance a document to the next stage, reject it, or request a revision — each with a comment.
- Approver statistics (assigned, pending, approved, revision, rejected).

Administration (Admin)

- Manage all users; search and filter by role; change a user's role.
- View user statistics (totals by role).
- Configure approval workflows by assigning the Initial, Compliance, and Final approvers for each workflow type.
- Monitor approver workload across the team.

Collaboration & traceability

- Comment threads attached to each document.
- Full, time-stamped approval history/timeline for every document.
- Multi-stage workflow (Initial → Compliance → Final) with per-stage approvers.
- Notifications for relevant events (*planned — endpoints are defined; persistence not yet added*).

6. Team Member Responsibilities

Use the table below to record each contributor and what they owned on the project. (Placeholders are provided for you to complete.)

Team Member	Responsibilities
Harshiya Saxena	Build the React + Vite single-page app; implement the role-based dashboards (Submitter, Approver, Admin), document views, upload/resubmit forms, and status tracking; handle client-side routing and styling.
Sangeetha S	Develop the Express server; implement authentication, JWT handling, and role-based access-control middleware; build the multi-stage approval workflow logic and file-upload handling.
Arshdeep S Virdi	Design and maintain the schema and data models (users, documents, workflows, comments, approval history); write queries, manage persistence, and handle seeding/migrations.
Rithika G	Design the REST endpoints and connect the frontend to the backend; define request/response contracts, wire up data flows, and manage error handling across the API layer.
Srujan P	Produce the project and API documentation; write test cases, fixtures, and automated/manual tests; run QA and verify features against requirements.

Tip: the GitHub repository lists the contributors and their commit history, which you can use to fill in this section accurately.

7. Setup & Deployment

The project is organized as a single repository (monorepo) containing both the Express backend and the React + Vite frontend, orchestrated from a root package.json. The same scripts cover installation, development, production builds, and starting the server.

7.1 Prerequisites

- Node.js and npm installed on the machine.
- A running instance of the project's database, reachable from the backend.
- Environment variables configured for the backend (for example the server port, the database connection string, and the JWT secret). Refer to the backend configuration for the exact variable names.

7.2 Project structure

```
document-approval-system/
├── backend/           # Express REST API (serves /api/*)
├── frontend/          # React + Vite single-page app
├── package.json       # root scripts (install-all, dev, build, start)
└── package-lock.json
```

7.3 Installation

From the repository root, install dependencies for both applications in one step:

```
npm run install-all
```

This runs `npm install` in both the backend and frontend folders.

7.4 Running in development

During development the backend and frontend run together (using concurrently):

```
npm run dev
```

7.5 Building & running in production

For production, compile the React frontend and then start the integrated Express server, which serves both the compiled frontend and the API from a single process:

```
# 1. Compile the React frontend
npm run build

# 2. Start the integrated Express server
npm run start
```

Once started, the application is available at <http://localhost:5000>.

7.6 Root scripts reference

- **install-all** — installs dependencies for both the backend and the frontend.
- **dev** — runs the backend and frontend together for local development.
- **build** — builds the production React frontend.
- **start** — starts the Express server (which also serves the built frontend).

7.7 Deployment

A public deployment of the application is hosted on Vercel: <https://doc-flow-fawn.vercel.app>.

In the deployed build, the Express server serves the compiled single-page frontend and the REST API together, so the client and API share the same origin.

8. API Documentation

8.1 Overview & conventions

- All endpoints are served under the `/api` base path (for example, <http://<server-host>/api/auth/login>).
- Requests and responses use JSON, except document upload (multipart/form-data) and download (binary file).
- Timestamps are ISO-8601 strings (e.g. 2025-06-11T10:30:00Z).

- Path parameters are shown as :id (the numeric document or user identifier).
- List endpoints accept query filters such as ?search=, ?status=, ?category=, and ?role= where noted.

8.2 Authentication & authorization

The API uses token-based authentication. A successful login returns an access token together with the user's role. Clients must send this token on every protected request using the HTTP Authorization header:

```
Authorization: Bearer <token>
```

Authorization is role-based. Endpoints are restricted as follows:

- **Public:** register and login.
- **Any authenticated user:** current-user info, logout, and the document/comment views appropriate to their role.
- **Submitter:** upload and resubmit documents; view and download their own documents.
- **Approver:** approve, reject, or request revision on assigned documents; view assigned lists and stats.
- **Admin:** manage users and roles, configure workflows, and view organization-wide statistics.

List endpoints that are “scoped by role” automatically return only the data the caller is allowed to see (for example, a submitter sees their own documents, while an admin sees all).

8.3 Endpoint reference

Authentication

#	Method	Endpoint	Purpose	Access
1	POST	/api/auth/register	Create a new user (name, email, password)	Public
2	POST	/api/auth/login	Authenticate user; returns token + role	Public
3	POST	/api/auth/logout	Invalidate the current session/token	Authenticated
4	GET	/api/auth/me	Get the current logged-in user's info & role	Authenticated

Users (Admin)

#	Method	Endpoint	Purpose	Access
5	GET	/api/users	List all users (?search=, ?role= filters)	Admin
6	PUT	/api/users/:id/role	Change a user's role (Submitter/Approver/Admin)	Admin
7	GET	/api/users/stats	Counts of users by role	Admin

Documents

#	Method	Endpoint	Purpose	Access
8	POST	/api/documents	Upload a document (multipart: title, description, category, file)	Submitter
9	GET	/api/documents	List documents (scoped by role; ?search=, ?status=, ?category=)	Authenticated
10	GET	/api/documents/:id	Get a single document's details	Authenticated
11	GET	/api/documents/:id/comments	Get the comments on a document	Authenticated
12	POST	/api/documents/:id/comments	Add a comment to a document	Authenticated
13	GET	/api/documents/:id/history	Get the approval timeline/history	Authenticated
14	PUT	/api/documents/:id/resubmit	Resubmit after revision (optionally a new file)	Submitter
15	GET	/api/documents/:id/download	Download the actual document file	Authenticated
16	GET	/api/documents/stats	Status counts, scoped by role	Authenticated

Approvals (Approver)

#	Method	Endpoint	Purpose	Access
17	POST	/api/documents/:id/approve	Approve and advance to the next stage	Approver
18	POST	/api/documents/:id/reject	Reject the document	Approver
19	POST	/api/documents/:id/revision	Request a revision from the submitter	Approver
20	GET	/api/approver/assigned	List documents assigned to the current approver	Approver
21	GET	/api/approver/stats	Approver stats (assigned, pending, approved, revision, rejected)	Approver
22	GET	/api/approvers/workload	Per-approver workload counts	Admin

Workflows (Admin)

#	Method	Endpoint	Purpose	Access
23	GET	/api/workflows	List all workflow configurations	Admin
24	PUT	/api/workflows/:id	Update a workflow's approver assignments	Admin
25	GET	/api/approvers	List users with the Approver role (for assignment dropdowns)	Admin

Notifications (planned)

These endpoints are defined in the API plan; persistence is not yet added.

#	Method	Endpoint	Purpose	Access
26	GET	/api/notifications	Get the current user's notifications	Authenticated
27	POST	/api/notifications	Create a notification (triggered internally)	System
28	PUT	/api/notifications/:id/read	Mark a notification as read	Authenticated
29	PUT	/api/notifications/read-all	Mark all notifications as read	Authenticated
30	DELETE	/api/notifications/:id	Delete a notification	Authenticated

8.4 Request & response examples

Register — POST /api/auth/register

Request body:

```
{
  "name": "Harshiya",
  "email": "harshiya@example.com",
  "password": "*****"
}
```

Login — POST /api/auth/login

Request body:

```
{
  "email": "harshiya@example.com",
  "password": "*****"
}
```

Response (returns a token and the user's role):

```
{
  "token": "<jwt-access-token>",
  "user": {
    "id": 1,
    "name": "Harshiya",
    "email": "harshiya@example.com",
    "role": "Submitter"
  }
}
```

The exact response shape may differ slightly in the implementation; the essential fields are the access token and the role.

Current user — GET /api/auth/me

Response:

```
{
```

```
{
  "id": 1,
  "name": "Harshiya",
  "email": "harshiya@example.com",
  "role": "Submitter"
}
```

Upload a document — POST /api/documents

Request (multipart/form-data fields):

```
title:      Project Proposal
description: Proposal for project
category:   Business
file:       <attached file>  (PDF, DOCX, or image)
```

Response:

```
{
  "id": 5,
  "title": "Project Proposal",
  "description": "Proposal for project",
  "category": "Business",
  "status": "Pending Approval",
  "version": "v1",
  "submittedBy": "Harshiya",
  "submittedAt": "2025-06-11T10:30:00Z"
}
```

List documents — GET /api/documents

Response (array; scoped by the caller's role):

```
[
  {
    "id": 1,
    "title": "Project Proposal",
    "status": "Pending Approval",
    "version": "v1",
    "date": "2025-06-10T10:30:00Z",
    "owner": "John Doe",
    "category": "Business"
  }
]
```

Document details — GET /api/documents/:id

Response:

```
{
  "id": 1,
  "title": "Project Name",
  "description": "Proposal for the new company project.",
  "workflowType": "Project Proposal",
}
```

```
"status": "Pending Approval",
"version": "v1",
"submittedBy": "John Doe",
"submittedAt": "2025-06-11T10:30:00Z",
"currentStage": "Initial Level",
"fileUrl": "/uploads/doc-1.pdf"
}
```

Approval history — GET /api/documents/:id/history

Response:

```
[
  { "id": 1, "step": "Submitted", "by": "John Doe",
    "time": "2025-06-11T10:30:00Z", "color": "#2563eb" },
  { "id": 2, "step": "Pending - Initial Level", "by": "System",
    "time": "2025-06-11T10:35:00Z", "color": "#f59e0b" },
  { "id": 3, "step": "Rejected at Initial Level", "by": "Sarah Khan",
    "time": "2025-06-11T11:00:00Z", "color": "#dc2626" }
]
```

Comments — GET /api/documents/:id/comments

Response:

```
[
  { "id": 1, "name": "Sangeetha",
    "text": "Please update the budget section before final approval.",
    "time": "2025-06-11T09:30:00Z" },
  { "id": 2, "name": "Admin",
    "text": "Document assigned to the department approver.",
    "time": "2025-06-11T08:30:00Z" }
]
```

Approve — POST /api/documents/:id/approve

Request body:

```
{
  "comment": "Looks good, approved for next stage."
}
```

Response:

```
{
  "id": 1,
  "status": "Pending Approval",
  "currentStage": "Compliance Level",
  "message": "Document advanced to Compliance Level"
}
```

Reject — POST /api/documents/:id/reject

Request body:

```
{
  "comment": "Budget section is incorrect, cannot approve."
}
```

```
}
```

Response:

```
{
  "id": 1,
  "status": "Rejected",
  "currentStage": "Initial Level",
  "message": "Document has been rejected"
}
```

Request revision — POST /api/documents/:id/revision

Request body:

```
{
  "comment": "Please update the budget numbers and resubmit."
}
```

Response:

```
{
  "id": 1,
  "status": "Revision Required",
  "currentStage": "Initial Level",
  "message": "Revision requested from submitter"
}
```

Change user role — PUT /api/users/:id/role

Request body:

```
{
  "role": "Approvers"
}
```

Response:

```
{
  "id": 1,
  "name": "John Doe",
  "email": "john@example.com",
  "role": "Approvers"
}
```

User statistics — GET /api/users/stats

Response:

```
{
  "totalUsers": 5,
  "totalSubmitters": 24,
  "totalApprovers": 8,
  "totalAdmins": 2
}
```

Document statistics — GET /api/documents/stats

Response (scoped by role):

```
{
  "total": 20,
  "approved": 9,
  "pending": 6,
  "revision": 3,
  "rejected": 2
}
```

Workflows — GET /api/workflows

Response:

```
[
  { "id": 1, "name": "Project Proposal",
    "initial": "Sangeetha", "compliance": "Rithika", "final": "Srujan" },
  { "id": 2, "name": "Budget Request",
    "initial": "Arshdeep", "compliance": "Sangeetha", "final": "Srujan" }
]
```

Update a workflow — PUT /api/workflows/:id

Request body:

```
{
  "initial": "John Doe",
  "compliance": "Rithika",
  "final": "Srujan"
}
```

8.5 Reference values

Roles

- Submitter — creates and resubmits documents.
- Approver — reviews assigned documents and approves / rejects / requests revision.
- Admin — manages users, roles, and workflow configurations.

Document statuses

- Pending Approval — awaiting action at the current stage.
- Approved — cleared all required stages.
- Rejected — declined by an approver.
- Revision Required — returned to the submitter for changes.

Workflow stages

- Initial Level → Compliance Level → Final Level, each handled by its assigned approver.